GPIO Input/Output with Raspberry Pi Zero

LAB 2

Group Members: Dipen Gaihre and Lotus Nyaupane

Course: Controllers and Electronics

Date: 6.11.2025

Table of Contents:

1. Objectives 3
2. Theory 3
3. Equipment and Materials 3
4. Methods 4
   A. Wiring the LED (Output) 4
   B. Writing Code for LED Output (Task 1 & Task 2) 5
   C. Wiring the Buttons (Input) 6
   D. Reading Button Presses (Task 3) 7
   E. Combining LED and Buttons (Task 4) 8
5. Results and Observations 9
6. Conclusion 10
References 11

1. Objectives

The main goal of this lab was to learn how to use Raspberry Pi Zero 2W for basic input and output tasks.

We practiced connecting an LED to a GPIO pin (output) and reading button presses (input).
At the end, we combined both to make the LED react to the button.

This lab helped us understand how microcontrollers talk to the outside world using GPIO pins.

2. Theory

Microcontrollers usually do two main things:

- **Output:** They can turn something on, like an LED.
- **Input:** They can read information, like button presses or sensor values.

GPIO (General Purpose Input Output) pins let us do both jobs.

LED Output
An LED needs a GPIO pin, a resistor, and a ground connection.
The resistor protects the LED and prevents it from burning.

Button Input
Buttons are simple switches. When pressed, the GPIO pin reads a change in voltage.
In this lab we used the pigpio library, which allows us to control and read GPIO pins in Python.

3. Equipment and Materials

- Raspberry Pi Zero 2W
- Grove Base Hat
- Grove Dual Button
- One LED
- 100 Ω Resistor
- 40-pin cable

- Breadboard
- Jumper wires
- USB power cable
- Python with pigpio library

4. Methods

A. Wiring the LED (Output)

We first connected the 40-pin flat cable to the Raspberry Pi Zero and then to the breakout board.

Then we built a simple LED circuit:

- GPIO pin → Resistor → LED → Ground

This allowed us to control the LED directly through Python.

Here is the picture from the LAB:

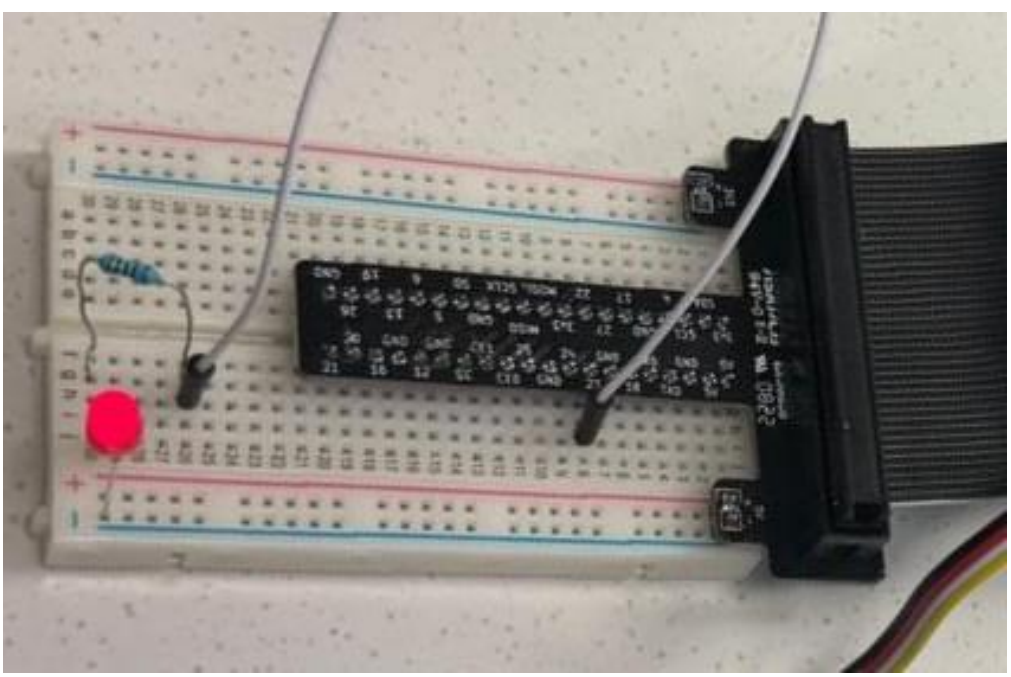


Fig: Led connected to breadboard

B. Writing Code for LED Output (Task 1 & Task 2)

We used the pigpio library to:

- Set the LED pin as OUTPUT
- Turn the LED ON and OFF
- Make the LED blink using the time.sleep() function

LED ON/OFF code (task 1) :

```
import pigpio
import time

pi = pigpio.pi()
if not pi.connected:
    print("NOT CONNECTED")
    exit()

LED_PIN = 23
pi.set_mode(LED_PIN, pigpio.OUTPUT)

pi.write(LED_PIN, 1)
print("LED ON")

time.sleep(3)

pi.write(LED_PIN, 0)
pi.stop()
print("Program finished.")
```

LED blinking code (task 2):

```
  GNU nano 8.4
import pigpio
import time

pi = pigpio.pi()
LED_PIN = 23

pi.set_mode(LED_PIN, pigpio.OUTPUT)

for i in range(5):
    pi.write(LED_PIN, 1)
    print("LED ON")
    time.sleep(1)

    pi.write(LED_PIN, 0)
    print("LED OFF")
    time.sleep(1)

pi.stop()
print("LED blinking finished automatically.")
```

C. Wiring the Buttons (Input)

We connected the Grove Dual Button to the Grove Base Hat.
The picture helped us match the correct GND and signal pins.
The buttons were connected to GPIO16 and GPIO17.

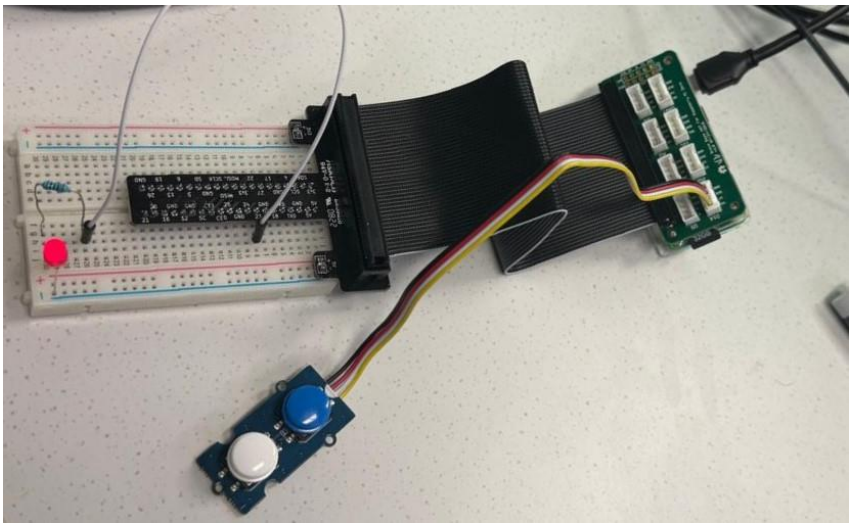


Fig 2: Buttons connected to Raspberry Pi

D. Reading Button Presses (Task 3)

We wrote Python code to set the button pins as INPUT and print messages when the buttons were pressed.

Code:

```
import pigpio
import time

pi = pigpio.pi()
BUTTON_PIN = 16


pi.set_mode(BUTTON_PIN, pigpio.INPUT)


pi.set_pull_up_down(BUTTON_PIN, pigpio.PUD_UP)

print("Reading button state 10 times...")

for i in range(10):
    state = pi.read(BUTTON_PIN)
    if state == 0:
        print(f"Reading {i+1}: Button pressed!")
    else:
        print(f"Reading {i+1}: Button not pressed.")
    time.sleep(1)

pi.stop()
print("Done reading button.")
```

E. Combining LED and Buttons (Task 4)

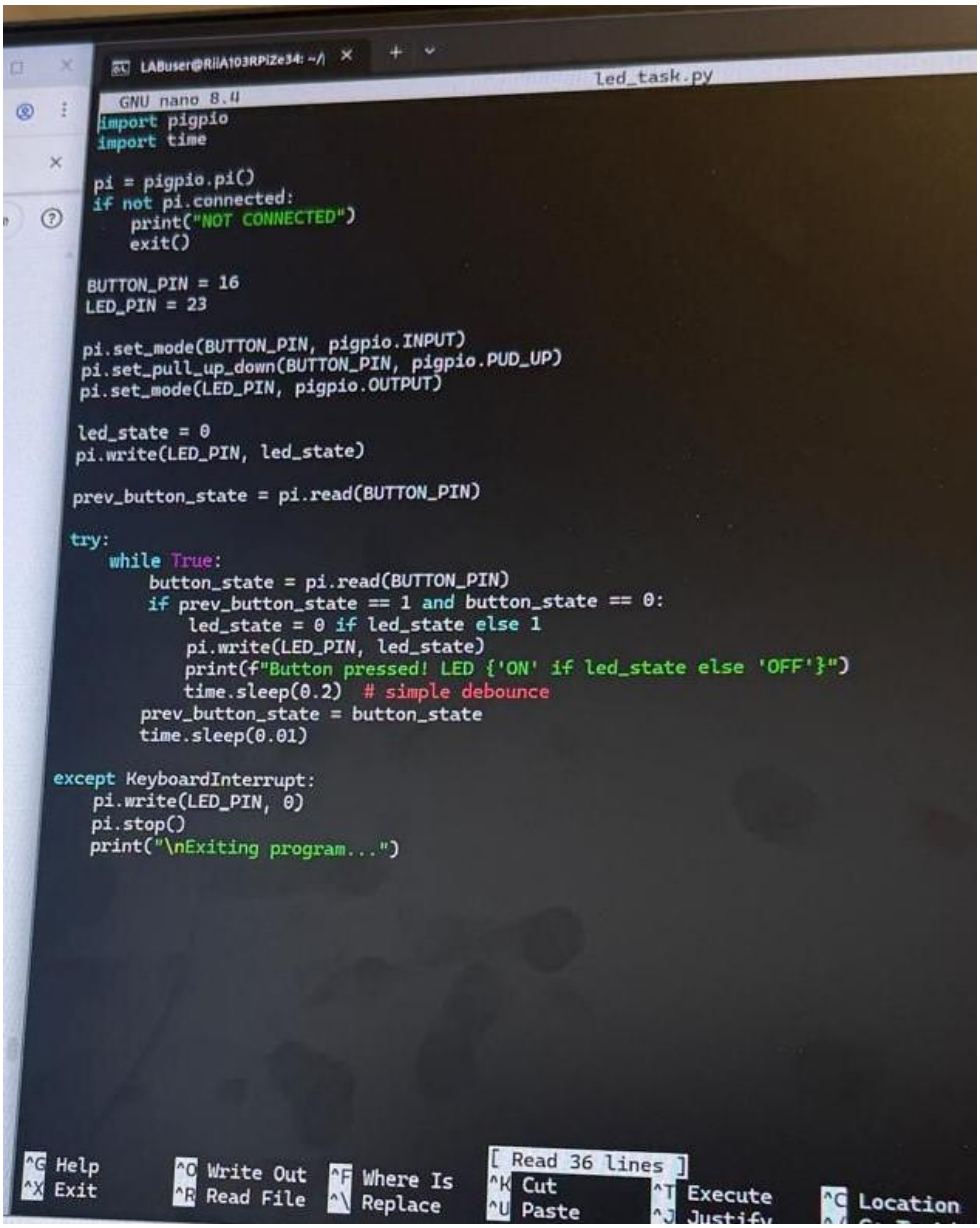
Finally, we combined everything:

- When Button A was pressed → LED turned ON
- When Button B was pressed → LED turned OFF
  (or any interaction you choose)

This showed how microcontrollers can use both INPUT and OUTPUT together.

Code:

Task 4: Combined LED + Button interaction code here



We also took a screenshot of the terminal:

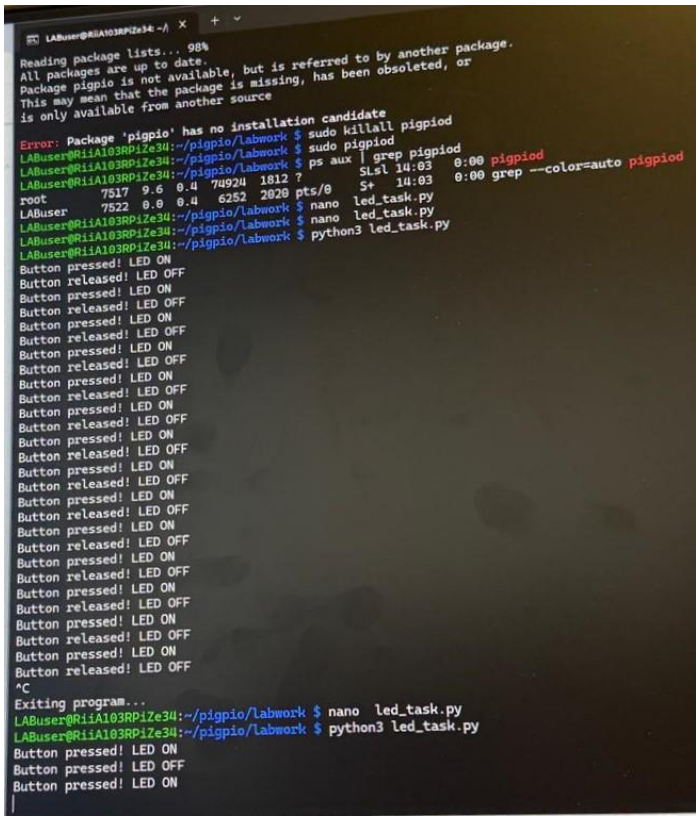


Fig: Terminal showing button presses

5. <u>Results and Observations</u>

- The LED wiring worked correctly, and we were able to control it using Python.
- The button input also worked, and the terminal printed the button press messages.
- When we combined both tasks, the LED reacted immediately to the button presses.

Things we noticed during the LAB:

- A correct grounding is important, otherwise nothing works.
- Pigpio requires running the sudo pigpiod command before the code.
- Sometimes the buttons bounced, meaning they registered multiple quick presses, which is normal.

Overall, everything behaved as expected.

6. Conclusion

In this lab, we learned how to use GPIO pins on the Raspberry Pi Zero for both input and output. We wired an LED, controlled it with Python, read button inputs, and finally made the LED respond to the buttons.

This lab gave us practical hands-on experience with microcontrollers and made GPIO concepts much clearer.
It also prepared us for future labs where sensors and more complex components will be used.

Note: we didn't find our Raspberry pi (29) so, we did project with another Raspberry pi (34).

References

- Pigpio Documentation
- Class Lectures and Instructions
- Help from teachers, classmates, and AI tools like ChatGPT